

Modern Dev Tools — Quadruple-Pass Audit

Performance · caching · IMGUI/VRAM · mod interoperability · new capabilities
Target: `asteryl.ModernDevTools v1.2` · 49 C# files, 8,797 LOC · RimWorld 1.6 / Unity Mono net472
Method: full static read of every source file, cross-checked against decompiled `Assembly-CSharp`, `HugsLib.dll` and `ArchotechLogs.dll`. **No changes made.** Nothing measured at runtime.

Headline. The architecture is sound — the analysis pipeline is correctly cached (`ConditionalWeakTable` per message), the index builds are prewarmed off-thread, and row culling already exists in the log list. The problems are concentrated in three places: **(1)** per-frame recomputation of things that never change while a message stays selected; **(2)** the knowledge base has no viewport culling at all; **(3)** replacing the vanilla log window silently deletes the log-window UI of *HugsLib* (active, load order #19) and *Archotech Logs*. Finding R1 is, in my judgement, the single most important item in this document — larger than any performance item.

On VRAM: there is essentially nothing to reclaim. Verified below — the entire suite draws from one shared engine texture. The right target is *CPU-side IMGUI work*, not GPU memory.

Certainty legend

Score	Means	Basis
95–99	Verified in source	I read the exact code path in this mod and/or decompiled engine/mod source. Non-speculative.
75–94	Strongly inferred	Code path verified; the <i>cost</i> or <i>consequence</i> is inferred from documented engine behaviour, not measured.
50–74	Reasoned estimate	Mechanism plausible and consistent with the code, but depends on runtime behaviour I did not observe.
<50	Needs a spike	Would require a prototype or a decompile I did not complete before committing.

Executive summary — ranked

#	Finding	Class	Why it ranks here	Cert.
R1	Replacing the log window silently removes HugsLib's log-window widget API (Share logs / Files / Copy) and every third-party widget registered through it	Compat	HugsLib is active at #19. <code>LogWindowExtensions</code> is the de-facto community standard for this. Total, silent feature loss, no error.	98
R2	Archotech Logs loses its Diagnostics/Export/colorizer UI entirely; its main-button fallback opens a <i>second, divergent</i> log window	Compat	Its whole UI hangs off <code>EditWindow_Log</code> postfixes we prevent from ever running.	97
P5	Knowledge base has zero viewport culling ; every filtered row draws and measures every pass	Perf	Harmony tab draws up to 300 cards × ~5 <code>Text.CalcHeight</code> each, 2–3 passes/frame.	98
P1–P4	Four per-frame recomputations of message-stable data (glossary regex sweep, impact assessment, stack-trace <code>CalcHeight</code> , rail counts)	Perf	All four are pure functions of the selected message. All four re-run on every OnGUI pass.	96
B4	Five hardening patches installed unconditionally for a default-off feature, incl. a finalizer on <code>Window.WindowOnGUI</code>	Perf	Full Harmony dispatch on the hottest UI method in the game, to do nothing.	96
N1	Yes, repeating log entries can be intercepted — and vanilla's dedupe has a real gap we can close	Feature	<code>StackTraceUtility.ExtractStackTrace()</code> runs <i>before</i> the dedupe test; interleaved spam never dedupes at all.	92
B1	<code>KnownIssueIndex.EnsureBuilt</code> publishes an empty list then fills it, unlocked, while the prewarm thread races it	Bug	Main thread can read a partial library → silently no diagnoses.	90
B2	<code>SettingsPage.Draw</code> has no <code>try/finally</code> around <code>BeginScroll / EndScroll</code>	Bug	One throw leaves the <i>global</i> <code>GUI.skin</code> scrollbar swapped for the rest of the session.	97
V1	VRAM is not a viable lever; draw-call count is	IMGUI	Every box is one <code>GUI.DrawTexture</code> of the shared <code>BaseContent.WhiteTex</code> . Mod-owned VRAM ≈ a few KB.	97
A1	Modder entry point is real but thin: no API version, no knowledge-source hook, no window-ownership contract	API	Third parties currently cannot extend the two things they most want to extend.	95

Pass 1 — Caching, polling, duplication

1.1 Per-frame work that should be cached

Everything in this table is a pure function of the *selected message* or of data that cannot change mid-session, yet is recomputed on every OnGUI pass. RimWorld runs 2–3 OnGUI passes per frame (Layout, Repaint, plus each input event), so multiply every number by that.

#	Site	What repeats per pass	When it bites	Cert.
P1	<code>Window_KnowledgeBase</code> → <code>Glossary.AllTerms()</code>	Allocates a fresh 26-entry <code>List<KeyValuePair></code> and performs 52 .Translate() calls . Called from <code>RailCount</code> , again from <code>DrawSources</code> , again from <code>DrawGlossary</code> — up to 3× per pass, purely to render a count badge.	KB open (always)	97
P2	<code>Module_Glossary.DrawSection</code>	<code>BuildText()</code> concatenates message + every diagnosis into one large string, then 26 compiled regexes sweep it. Rebuilt from scratch every pass.	Any message selected	96
P3	<code>ImpactAssessor.Assess</code> via <code>DrawImpactBanner</code>	Up to 49 IndexOf scans over the full stack trace + message (19 tick signals × 2 + 11 frame signals), plus a LINQ <code>.Any()</code> , plus <code>.Translate()</code> string building for the perf note.	Any message selected	96
P4	<code>Window_ModernLog.DrawInspectorContent</code>	<code>Text.CalcHeight()</code> on the entire stack trace and on the full message body. Unity text measurement on a 40–80 line string is the single most expensive call in the inspector.	Any message selected	95
P5	<code>Window_KnowledgeBase</code> — all 6 list tabs	No viewport culling . <code>DrawLibrary</code> , <code>DrawCommunityBugs</code> , <code>DrawHarmony</code> , <code>DrawGlossary</code> , <code>DrawDependencies</code> , <code>DrawCompatibility</code> each iterate the whole filtered list and call <code>DrawIssueCard</code> , which does 5 Text.CalcHeight calls per card — even for cards scrolled far off screen. Harmony tab caps at 300 rows → ~1,500 measurements/pass.	KB open, any list tab	98
P6	<code>RebuildFiltered</code> → <code>KnownIssueIndex.TextMatchesAnyIgnored</code>	Per message: <code>text.ToLowerInvariant()</code> (a full string copy) and a regex-driven <code>ExtractExceptionType</code> . With 1,000 queued messages that is 1,000 allocations + 1,000 regex runs per rebuild , and a rebuild is triggered by every single enqueued log line.	Log open + any ignore set + spam	94
P7	<code>Window_ModernDevPalette.Draw</code>	<code>ResolveNodes()</code> calls <code>Dialog_Debug.GetNode(path)</code> <i>per pin, per frame</i> . Each call does <code>string.Split</code> , then per path segment a <code>FirstOrDefault</code> with a capturing closure (closure + delegate allocation) plus <code>TrySetupChildren()</code> . <code>DesiredWidth()</code> additionally re-measures every label in two fonts every Repaint.	Palette open	93
P9	<code>Window_KnowledgeBase.CountDefs</code>	Full scan of <code>DefDatabase<KnownIssueDef></code> ×4 per pass (twice in the rail, twice in Sources).	KB open	97

Already correct — do not "fix" these. The log list *does* cull rows (`first / last` window in `DrawList`). The spawn grid culls rows. `LogAnalysisCache` is a well-chosen `ConditionalWeakTable` that auto-evicts with the message. `RebuildFiltered` is correctly revision-gated. `Compat()` is stamp-gated. Index prewarm is off-thread. Scroll heights are floored at viewport height, avoiding the known "short-frame latch" trap. `Palette.Accent` is frame-memoised.

1.2 Analysis-pipeline waste (per analysis, not per frame)

#	Site	Issue	Cert.
P8	<code>ErrorContext.Culprits</code>	Implemented as <code>RankedMods().Where(...)</code> , and <code>RankedMods()</code> performs a five-key LINQ OrderBy chain plus ToList() on every access. Four modules (<code>Community</code> , <code>DependencyCheck</code> , <code>AboutIncompat</code> , <code>VersionMismatch</code>) each <code>foreach</code> it → four full sorts and four list allocations per analysed message. Rank once, cache, expose a read-only view.	97
P10	~100 <code>RegexOptions.Compiled</code> patterns	44 library defs (+ attribute regexes), 26 glossary, 17 community, 13 static. On Unity Mono , <code>Compiled</code> emits IL through <code>Reflection.Emit</code> at construction — a real startup cost, and the generated dynamic methods are never reclaimable. For short patterns matched a handful of times, interpreted regex is usually cheaper overall. Worth benchmarking before assuming <code>Compiled</code> is a win. Keep it for the hot text-scan patterns only.	75
P11	<code>foreach (LogMessage m in Log.Messages)</code>	<code>Log.Messages</code> is declared <code>IEnumerable<LogMessage></code> , so the <code>Queue<T></code> struct enumerator is boxed on each rebuild. Two sites (<code>RebuildFiltered</code> , <code>BuildAllMessages</code>). Minor, but free to fix.	90

#	Site	Issue	Cert.
P12	FrameParser.TypeCache	Static Dictionary<string,Type>, unbounded and unsynchronised. Grows for the whole session with every distinct qualified type name seen. Cap it, or accept and document.	92

1.3 Duplicated code

Duplicate	Locations	Suggested home	Cert.
Unordered pair key	Window_KnowledgeBase.PairKey, AboutIncompatIndex.PairKey, Module_AboutIncompat.PairKey — three byte-identical copies	ModIdUtil.PairKey	99
Issue-scoring algorithm	KnownIssueIndex.Match and CommunityData.Match implement the same weights (exType 3 / regex 3 / keyword 2 / namespace 2 / packageId 2) over two different data shapes. A weight change must be made twice or the surfaces silently diverge.	One IssueMatcher over a shared IMatchSignals	98
Module row rendering	SettingsPage.DrawModuleRow vs the inline loop in Dialog_Modules.DrawModules — ~20 identical lines. This is exactly the drift the UI-Parity contract exists to prevent.	Palette.ModuleRow	97
Checkbox glyph	Window_ModernDevActions.DrawCheck and Window_ModernDevPalette.DrawCheck (differ only by 1px padding)	Palette.DrawCheck	98
Dead chevron	Window_ModernDevActions.DrawChevron is private and never called — the class uses Palette.DrawChevron.Delete.	—	96
Regex/lowercase helpers	Lower() and CompileRegexes() exist in both KnownIssueIndex and CommunityData; StripBom in both CommunityData and ModShippedIssues	TextUtil	98

1.4 "Missing libraries"

I checked for genuinely absent infrastructure rather than assuming a package is needed.

- **JSON:** the hand-rolled Source/Community/Json.cs (146 LOC) is the *correct* call. RimWorld ships no JSON library and adding Newtonsoft would mean shipping a DLL and inviting assembly-version conflicts. Keep as-is
- **Harmony:** correctly referenced with ExcludeAssets="runtime" and declared as a mod dependency — 0Harmony.dll is not shipped. Correct
- **Genuinely missing (internal):** a shared TextMetrics memo cache keyed on (string, width, font, fontSize, wordWrap). Its absence is the direct cause of P1–P5. This is the one piece of "framework" the mod actually lacks. Build it
- **Genuinely missing (internal):** a per-analysis derived-data cache hanging off LogAnalysis (impact result, glossary terms, measured heights). Also the direct cause of P2–P4. Build it

1.5 "Multi-polling ticks"

Clean. The mod registers *no* GameComponentTick, MapComponentTick or ThingComp tick. GameComponent_ModSnapshot is event-driven, not polled. There is no tick-rate cost at all when no window is open. The per-frame cost is entirely OnGUI-scoped and therefore only paid while a dev window is visible — which is the right design. The one exception is B4 below: three Harmony hooks that *do* run every frame regardless.

1.6 Correctness and robustness defects

#	Site	Defect	Cert.
B1	KnownIssueIndex.EnsureBuilt	Publishes _all = new List<Compiled>() then fills it, with no lock. The startup prewarm runs this on a background thread. A main-thread caller arriving mid-build sees _all != null, returns immediately, and iterates a list being mutated → either a swallowed InvalidOperationException (the try/catch in Match hides it) or a partial library. Symptom: an error inspected right after launch silently gets no diagnoses. Build into a local, assign under a lock — exactly the pattern HarmonyIndex.Build() already uses correctly.	90
B2	SettingsPage.Draw	No try/finally around Palette.BeginScroll / EndScroll. Every other call site pairs them correctly. If DrawContent throws, GUI.skin.verticalScrollbar* stays swapped globally and BeginScrollView is left unbalanced — corrupting every vanilla scrollbar for the rest of the session. Compounded by the known engine behaviour that a throwing window is force-closed by WindowStack, so the user sees settings vanish and every scrollbar go flat.	97
B3	Palette.BeginScroll	Saves the previous skin into single static fields, not a stack — it is not re-entrant. Today no call site nests, so it works. But ErrorModule.DrawSection is invoked <i>inside</i> the inspector's active scroll view, so the first third-party module that calls Palette.BeginScroll corrupts the restore. This is an API-surface hazard, not just a latent bug.	95

#	Site	Defect	Cert.
B4	5 hardening patches	Both hardening features default to off , yet the patches are always installed: a finalizer on <code>Window.WindowOnGUI</code> (runs for every window, every pass), a postfix on <code>WindowStack.WindowStackOnGUI</code> (every frame), a postfix on <code>Log.Error</code> (every error in the game), and three map/world OnGUI finalizers. A guard clause makes the body free; it cannot make the <i>dispatch</i> free, and a finalizer additionally forces the target into a try/catch. Move to settings-driven <code>Patch / Unpatch</code> , or at minimum a <code>Prepare()</code> gate.	96
B5	<code>CommunityData</code> background thread	Parsed dictionaries are published through <code>volatile</code> reference assignment — correct. But the finally block calls <code>LogAnalysisCache.Clear()</code> off-thread, replacing a static field a main-thread <code>GetValue</code> may be using. Benign in practice (worst case one stale analysis) but worth a main-thread marshal for hygiene.	70

Pass 2 — ImGui usage, and the VRAM question answered

Direct answer: there is no meaningful VRAM to save, and no lower-VRAM alternative exists that preserves the aesthetic. The premise does not hold for this mod — and that is good news.

2.1 What the suite actually costs on the GPU

Verified against decompiled Verse.Widgets:

```
// Verse/Widgets.cs:827 - the primitive behind every solid fill in the suite
public static void DrawBoxSolid(Rect rect, Color color) {
    Color prev = GUI.color;
    GUI.color = color;
    GUI.DrawTexture(rect, BaseContent.WhiteTex); // ONE shared texture, tinted by GUI.color
    GUI.color = prev;
}

// Verse/Widgets.cs:844 - a 1px border is FOUR separate draws of that same texture
public static void DrawBox(Rect rect, int thickness = 1, Texture2D lineTexture = null) {
    GUI.DrawTexture(...left...); GUI.DrawTexture(...right...);
    GUI.DrawTexture(...top...); GUI.DrawTexture(...bottom...);
}
```

Consequences, all verified:

- Colour is a GUI.color tint, **not** a texture. The palette's ~14 colours allocate zero textures.
- Total GPU residency owned by this mod: 3 shipped icons (Pin, Chevron, Close) plus **three 2×2 textures** created in Palette.InitScroll via SolidColorMaterials.NewSolidColorTexture. That is a few kilobytes. There is no atlas, no render texture, no material churn, no per-frame allocation.
- Therefore every "reduce VRAM" technique — atlasing, compression, texture pooling, render-texture caching — targets something that does not exist here. Two of them would *increase* VRAM.

The real cost is Palette.DrawCard = 5 draw calls (1 fill + 4 border edges), and the card is the mod's dominant primitive. Static count across the codebase: 101 call sites of DrawBoxSolid / DrawBox / DrawCard / DrawWell.

2.2 Estimated per-pass draw budget

Counted statically from the draw code for a representative log window (20 visible rows, 4 diagnoses). This is an estimate from code reading, not a profiler capture.

Region	Composition	GUI.DrawTexture calls
Window shell	fill + border	5
Toolbars (3 chips, 3 toggles, 5 buttons, search)	6 + 6 + 5 + 5 each	~66
Message list (well + 20 rows)	5 + 2/row	~45
Inspector chrome (card, banner, 3 buttons, 2 wells)	5 + 6 + 15 + 10	~36
2 source rows + 4 diagnosis cards + report card + glossary card	6 each	~48
Total per OnGUI pass (×2–3 per frame)		~200–260

2.3 Alternatives evaluated — with verdicts

Technique	Assessment against "aesthetic and function cannot change"	Verdict	Cert.
UI Toolkit / UIElements	RimWorld's entire Window pipeline is ImGui (GUI.Window + GUIClip). There is no hook to host a UIElements panel inside a Window, and mixing the two breaks clipping, hit-testing and z-order. Would also require rewriting every widget.	Reject	95
Render-texture caching of static panels	Would add VRAM (one RT per panel), and breaks hover states, tooltips and text-field focus — a function change. The panels also change content constantly.	Reject	90
Batching quads via GL / Graphics.DrawMeshNow	Technically feasible and pixel-identical, but immediate GL inside nested GUIClip groups is exactly the class of thing that fails silently at non-1.0 UI scale. High risk, unquantified payoff.	Not now	65

Technique	Assessment against "aesthetic and function cannot change"	Verdict	Cert.
Two-draw card: draw the border colour as one filled rect, then the fill colour inset by 1px	Pixel-identical 1px ring at UI scale 1.0, and cuts <code>DrawCard</code> from 5 draws to 2 (~60% on the dominant primitive; whole-window budget ~260 → ~170). <i>Caveat:</i> <code>Widgets.DrawBox</code> snaps each edge with <code>UIScaling.AdjustRectToUIScaling</code> ; the two-draw form does not, so it must be verified at 125% and 150% UI scale before adoption.	Prototype	85
Memoised text metrics	Not a render change at all — and by every available account the dominant <code>IMGUI</code> cost is Unity text measurement, not quad submission. Zero visual risk.	Do first	94
Viewport culling in the KB	<code>BeginScrollView</code> clips pixels but does <i>not</i> skip draw code. Culling is invisible to the user and removes the largest single block of work in the mod.	Do first	98
Reducing <code>StateStrip</code> / hover overlays	Each is 1 draw. Removing them would change the aesthetic. Leave alone.	No change	99

Conclusion for Pass 2. "Run better than vanilla with less overhead" is achievable, but the route is *caching and culling*, not GPU memory. Vanilla's `EditWindow_Log` is a plain `Listing` over the message queue — it is cheap because it does very little. Our window does much more, so parity-or-better means paying for the extra features *once per selection* rather than once per pass. Items P1–P5 alone are, by my reading, the difference.

Pass 3 — Mod interoperability (deep scan)

3.1 Complete inventory of external touch points

Class	Surface	Risk
Harmony patches	16 targets. <code>UIRoot.CheckOpenLogWindow</code> , <code>DebugWindowsOpener</code> ×4, <code>EditWindow_Log.SelectLastMessage</code> , <code>DebugActionNode.Enter</code> , <code>LogMessageQueue.Enqueue / Clear</code> , <code>Window.WindowOnGUI</code> , <code>Log.Error</code> , <code>WindowStack.WindowStackOnGUI</code> , <code>WorldInterface</code> , <code>MapInterface</code> ×2, <code>GlobalControls</code>	High — 5 are <code>Prefix + return false</code>
Private-field reflection	<code>Dialog_Debug.roots</code> , <code>EditWindow_Log.canAutoOpen</code> — both bound once via <code>AccessTools</code> , both null-guarded with a logged warning. Well done	Low
Reflection into other mods	<code>ModernNotifications.Theme.Accent</code> (suite accent bridge), <code>Circinus.KnownIssues</code> (corpus). Both probe-once, cache, null-every-binding-on-failure, and warn loudly only on <i>contract drift</i> . This is the right pattern — generalise it	Low
Reading other mods' data	<code>About.xml</code> (<code>incompatibleWith</code> , <code>dependencies</code> , <code>supported versions</code>), <code>ModsConfig</code> load order, <code>About/known-issues.json</code> shipped files, the live Harmony registry, and every loaded mod assembly's type namespaces	Low — all read-only
Network	3 endpoints, opt-in, background, cached, offline fallback	Low

3.2 R1 HugsLib — the biggest compat liability

HugsLib is **active in this modlist at load order #19**. Decompiling `HugsLib.Logs.LogWindowExtensions` confirms it exposes a **public, XML-documented extension API** that is the community's de-facto standard for adding buttons to the debug log:

```
public static void AddLogWindowWidget(WidgetDrawer drawerDelegate,
                                     WidgetAlignMode align = WidgetAlignMode.Left);

public delegate void WidgetDrawer(Window logWindow, Rect widgetArea,
                                  LogMessage selectedLogMessage, WidgetRow widgetRow);
```

It is rendered by `HugsLib.Patches.EditWindow_Log_Patch`, a **prefix on** `EditWindow_Log.DoMessagesListing`.

What breaks. Modern Dev Tools prefixes `UIRoot.CheckOpenLogWindow` and `DebugWindowsOpener.ToggleLogWindow` and returns `false` unconditionally. The auto-open path, the dev-toolbar button and the hotkey are the *only* ways vanilla opens `EditWindow_Log`. With our mod active, **`EditWindow_Log` is never constructed**, so `DoMessagesListing` never runs, so `DrawLogWindowExtensions` never runs. Silently lost:

- HugsLib's **"Share logs"** button (`LogPublisher`) — the single most-used log feature in the RimWorld support ecosystem. Every "post your HugsLib log" support request now fails.
- HugsLib's **"Files"** menu (open `Player.log` / save dir / mods dir).
- HugsLib's **"Copy"** button.
- **Every third-party widget** any other mod registered through `AddLogWindowWidget`.

No error, no warning, no message. The user simply concludes HugsLib broke. This is doubly notable because the Schematic records a deliberate decision *not* to clone HugsLib's log upload — correct on its own terms, but it assumed HugsLib's own button would still be reachable. It is not.

Certainty 98

Proposed fix — a HugsLib widget host (tiered)

Tier	Approach	Aesthetic cost	Cert.
T0	Escape hatch. When HugsLib is active, add one suite-styled <i>"Open vanilla log"</i> button to Toolbar B that constructs <code>EditWindow_Log</code> directly. Restores 100% of HugsLib functionality for zero risk and zero reflection.	None — one more gray button	96
T1	Host their widgets. Reflect the private static <code>LogWindowExtensions.widgets</code> list, read each entry's public <code>Drawer</code> and <code>Alignment</code> fields, and invoke the delegate with <i>our</i> window (the parameter is typed <code>Window</code> , so we qualify), our selected <code>LogMessage</code> , and a real <code>WidgetRow</code> . Draw them into a dedicated, labelled <i>"Add-ons"</i> strip at the bottom of the inspector.	Foreign buttons use vanilla <code>WidgetRow</code> styling. Containing them in their own strip keeps suite chrome uncontaminated — this is a deliberate, defensible boundary rather than a style break.	88
T2	Reciprocal API. Ship our own <code>RegisterToolbarWidget</code> (see A1) and host HugsLib's. Then we are a superset, and we can invite HugsLib-targeting mods to register natively for full suite styling.	None	85

T0 should ship regardless — it is a handful of lines and it converts a silent total failure into a one-click detour. T1 is the quality answer.

3.3 R2 Archotech Logs — requested compat

kongkim.ArchotechLogs (Workshop 3710696168, currently installed but inactive) is a direct functional neighbour: "a detailed log view and an advanced log export system". Decompiled, its entire UI hangs off the vanilla window:

Archotech class	Hook	Fate with MDT active
LogWindowPatch	Postfix EditWindow_Log.DoWindowContents — draws <i>Diagnostics, Export, ?</i> , colour-mode toggle	Never runs
LogWindowSpamOverlayPatch	Postfix same method — repeat-count / hyperactive-loop banner	Never runs
LogWindowCrashProtector	Finalizer on DoMessagesListing	Moot (no conflict with our watchdog — different method, ours off by default)
LogWindowSync_Patch	Postfix EditWindow_Log.PostClose	Moot
MainButtonWorker_LogShortcut	Activate() calls Find.WindowStack.Add(new EditWindow_Log()) directly	Bypasses our patches → a second log window opens with divergent state

Net user experience today: if their main-button setting ShowGlobalLogButton is on, they get two competing log windows. If it is off, Archotech Logs is completely unreachable and appears broken. Certainty 97

Proposed Archotech compat (tiered)

- A-T0 Detect and disclose. On first run with kongkim.ArchotechLogs active, show a one-time notice explaining both windows exist, and add a "Log window owner" setting: Modern / Vanilla (+ Archotech) / Both. Choosing Vanilla makes our prefixes return true, standing down cleanly. 96
- A-T1 Bridge the buttons. Add an Archotech group to the inspector's add-on strip that opens Dialog_ArchotechInfoView and ArchotechExportWindow, and mirrors LogColorizer.UseArchotechColors. Blocker: Dialog_ArchotechInfoView's constructor takes an EditWindow_Log. Two routes: (a) pass a detached, never-added EditWindow_Log instance purely as the ctor argument — only safe if the dialog never adds it to the stack or reads per-instance state, which I did not verify; (b) ask kongkim for a parameterless overload. Route (a): 45 — needs a decompile of the dialog before committing
- A-T2 Consume, don't duplicate. Archotech's SuspectAnalyzer, IssueClassifier, HarmonyTracker and ModMapper solve the same attribution problem we do. Rather than compete, ship a Module_ArchotechLogs : ErrorModule that reflects its classifier and surfaces its verdict as one more diagnosis with a low score — exactly the Module_CircinusCorpus pattern. Zero coupling, zero overhead when absent. 65

3.4 Structural interop weaknesses

#	Issue	Cert.
R3	All five window prefixes return false unconditionally, with no Harmony priority and no "has someone else already claimed this?" check. Two log-replacer mods would resolve non-deterministically by patch order. Declare an explicit [HarmonyPriority] and gate on a public ownership flag.	90
R4	No public "who owns the log window" contract. Other mods cannot detect that the vanilla window is suppressed, cannot ask us to open, and cannot decorate us. This is what makes R1 and R2 silent rather than adaptable.	95
R5	Patch_SelectLastMessage prefixes EditWindow_Log.SelectLastMessage and returns false. That method is also vanilla's own behaviour for the openOnMessage flow. If any path still shows the vanilla window (Archotech's main button does), we have broken its selection while not owning it.	92
R6	The Log.Error postfix (Patch_Log_Error_FillDetector) puts us on the error path of every mod in the game for a default-off feature — both a cost (B4) and an extra chance of ordering friction with other log-adjacent mods.	95
R7	Module_CircinusCorpus is an excellent compat pattern but hardcoded. Generalise it into a declarative reflection bridge so Archotech, HugsLib and future neighbours are configuration, not new classes.	94

3.5 "Can installed mods give us base-game-rooted features at near-zero cost?"

Scoped honestly: I surveyed the 909-mod list for the class that shares our patch surface rather than speculating across all of them. Only three are relevant, and all three argue for the same conclusion.

Mod	State	Opportunity	Cert.
HugsLib	Active #19	Host its widgets (R1-T1). Feels 100% base-game because it is the button users already know. Cost: one reflected list read at window open. This is the highest-value, lowest-overhead addition available.	96
Archotech Logs	Installed, inactive	Diagnostics/Export bridge + optional consumption of its classifier (3.3).	65

Mod	State	Opportunity	Cert.
Dubs Performance Analyzer	Installed, inactive	It owns the per-patch <i>calls-per-frame</i> data our Harmony tab can only guess at. A reflection bridge could show "this method is patched by 3 mods and costs X ms/frame" — genuinely new information, and exactly the evidence our impact scoring currently lacks. Gate on <code>requiresPackageId</code> , so zero cost when absent.	50 — API surface unverified

The generalisable answer is **R7**: rather than hand-writing a compat class per neighbour, build one declarative reflection-bridge module type. Then each new integration is a def, not a release.

Pass 4 — New capabilities

4.1 N1 Intercepting repeating log entries — yes, and there is a real gap to close

Verified against decompiled `Verse.Log` and `Verse.LogMessageQueue`. Four facts drive the design:

```
// Verse/Log.cs - Error(), Warning() and Message() all share this shape
lock (logLock) {
    messageQueue.Enqueue(
        new LogMessage(type, text, StackTraceUtility.ExtractStackTrace()), // (1) ALWAYS runs
        out var repeatsCapped); // even for repeat #500
    if (repeatsCapped) return; // (2) cap is 99
    ...
}
Debug.LogError(text); // (3) Unity -> synchronous Player.log write, until the cap

// Verse/LogMessageQueue.cs - (4) dedupe compares against lastMessage ONLY
if (lastMessage != null && msg.CanCombineWith(lastMessage)) { ... }
```

- 1. `StackTraceUtility.ExtractStackTrace()` is evaluated before the dedupe test. Every repeat pays a full managed frame walk plus a large string allocation. On Mono this is expensive, and it is unavoidable through vanilla's own path. `new LogMessage` additionally does `DateTime.Now.ToString(...)` per call.
- 2. The repeat cap is **99**. Below it, every occurrence also does a synchronous `Debug.Log*` → `Player.log` disk write.
- 3. All of it is inside `lock (logLock)`, so tick-loop spam serialises through one lock.
- 4. **The gap**: dedupe only compares against `lastMessage` — it is strictly *consecutive*. Two interleaved errors (A, B, A, B...) **never combine**, so they never reach the cap, never stop writing to disk, and permanently churn the 1,000-entry queue. This is the common real-world case: one broken pawn producing two errors per tick.

Proposed design — "Throttle repeating log spam" (toggleable)

```
Prefix on Log.Error(string) / Log.Warning(string) / Log.Message(string):

1. If the feature is off -> return true (vanilla, untouched)
2. key = FNV-1a(text) + type // no ToLower, no substring, zero allocation
3. Look key up in a fixed 64-slot ring of {hash, count, firstFrame, lastFrame}
4. count < Threshold (default 5) -> return true // first occurrences ALWAYS pass
5. count >= Threshold, within window -> count++; return FALSE
   skips: ExtractStackTrace, LogMessage alloc, Debug.Log*, Player.log write,
   TryOpenLogWindow, and the pause-on-error tick pause
6. Every N suppressed (or on a timer) emit ONE summary line so nothing vanishes silently
7. Surface "suppressed x N" on the message in our inspector + impact banner
```

Aspect	Assessment	Cert.
Mechanism works	A Harmony prefix returning <code>false</code> on <code>Log.Error(string)</code> demonstrably skips the whole body. <code>Log.ErrorOnce</code> routes through <code>Error</code> , so it is covered too.	92
Closes a real vanilla gap	The interleaved-spam case is genuinely unhandled by vanilla, and our ring buffer handles it. This is the part that is <i>new</i> , not just cheaper.	95
Magnitude of the win	Unmeasured. Removing stack extraction + a synchronous disk write per error <i>should</i> dominate during a storm, but I will not put a number on it without a profiler capture. Frame it to users as "removes the logging overhead of the symptom", never as "fixes the cause".	60
Risk: never hide the first one	Non-negotiable. The diagnostic value is the first occurrence plus an accurate count. Threshold must be ≥1 and the summary line mandatory.	99
Risk: other mods' postfixes	Anything postfixing <code>Log.Error</code> stops seeing suppressed calls — including our own <code>WindowWatchdog</code> detector. Must be documented, and the watchdog must read our counter instead.	94
Risk: we are on every error path	The prefix must be allocation-free, lock-free and exception-proof. A throw here would be catastrophic and self-concealing. Wrap in <code>try/catch</code> returning <code>true</code> (fail open to vanilla) on any fault, and latch off after repeated faults.	97
Interaction with pause-on-error	Suppression also skips <code>Find.TickManager.Pause()</code> . That is arguably desirable (no repeated pausing) but it is a behaviour change — must be called out in the setting's description.	95

4.2 A1 A robust open entry point for modders

What exists today is good and genuinely usable: `ErrorModuleDef` + `ErrorModule` (XML or code), `ModernDevToolsAPI.RegisterModule`, and the zero-dependency `About/known-issues.json` drop-in — that last one is an excellent piece of design, because it requires no reference, no `LoadFolders` and no C#. The gaps:

Gap	Proposal	Cert.
No version contract	<code>public const int ApiVersion = 2;</code> plus a frozen, commented contract block (the <code>Module_CircinusCorpus</code> doc-block style) so third parties can soft-bind by reflection and detect drift.	97
Cannot register a knowledge source	<code>RegisterKnowledgeSource(IKnownIssueSource)</code> — lets a mod supply issues dynamically (from its own settings, a server, or generated) rather than only a static JSON file. Feeds the same shared <code>IssueMatcher</code> from §1.3.	94
Cannot add UI	<code>RegisterToolbarWidget(id, drawer)</code> for Toolbar B and an add-on strip in the inspector — deliberately modelled on HugsLib's <code>WidgetDrawer</code> signature so a mod can register once for both. This is the same hook that solves R1.	90
No completion event	<code>event Action<LogAnalysis> AnalysisCompleted</code> — enables exporters, telemetry, and Archotech/Circinus-style consumers without a module.	93
No window-ownership query	<code>LogWindowOwner</code> (who owns it), <code>OpenLog()</code> , <code>IsLogOpen</code> , and a <code>YieldLogWindow()</code> stand-down call. Directly resolves R3/R4/R5.	95
Unsafe drawing contract	<code>ErrorModule.DrawSection</code> currently hands out a raw y-cursor inside an active scroll view, where a module calling <code>Palette.BeginScroll</code> corrupts global skin state (B3). Fix B3, then document the contract explicitly: "you are inside a scroll view; do not begin another".	95

Paths forward

Sequenced so that each phase is independently shippable and independently verifiable. Nothing here changes the aesthetic or the behaviour of any existing feature.

Phase	Work	Why this order	Effort	Risk	Payoff
0	Correctness first. B1 (lock the index build), B2 (try/finally), B3 (make BeginScroll re-entrant), B5 (marshal the cache clear)	These are latent bugs, not optimisations. Fix before touching anything they sit under.	Small	Very low	Stability
1	R1-T0 + A-T0. "Open vanilla log" button when HugsLib/Archotech is active, plus the log-window-owner setting and first-run notice	Converts two silent total-failure modes into a one-click detour. Handful of lines. Highest value per unit of risk in this whole document.	Small	Very low	Very high
2	Cache pass. Shared <code>TextMetrics</code> memo; hang <code>ImpactResult</code> , glossary terms and measured heights off <code>LogAnalysis</code> ; cache <code>RankedMods</code> ; cache KB rail counts; cache palette node resolution	Fixes P1–P4, P7–P9 with one shared mechanism. Pure win, zero visual change.	Medium	Low	Very high
3	Cull the knowledge base. Two-pass measure/draw with a cached per-row height, skipping rows outside the viewport	Largest single block of avoidable work in the mod. Independent of phase 2.	Medium	Low-med (layout)	Very high
4	B4. Move the 5 hardening patches to settings-driven <code>Patch / Unpatch</code>	Removes per-frame dispatch for a default-off feature. Must be driven from a tick/component context, never from inside a patch body.	Small-med	Medium	High
5	API v2 (§4.2) and R1-T1 (host HugsLib widgets) — same hook, ship together	The widget host is the API's first consumer, which proves the design.	Medium	Medium	Very high
6	N1 log-spam throttle, default <i>off</i> for one release, then default on	It sits on every mod's error path. Earn confidence before defaulting it on.	Medium	Med-high	High
7	Dedupe pass (§1.3) — unify the two issue matchers, hoist shared helpers, delete dead code	Best done after phases 2–5, so the refactor absorbs the new shapes rather than fighting them.	Medium	Low	Maintainability
8	R7 declarative reflection-bridge module; then A-T1/A-T2 and the Dubs bridge as defs	Turns every future integration from a code change into a def.	Medium	Medium	High
9	Two-draw card prototype, verified at 100/125/150% UI scale	Real but smallest win, and the only item with genuine visual-regression risk. Last, and only if the screenshots are identical.	Small	Medium (visual)	Medium

What I did not verify

Stated plainly so nothing here is mistaken for measurement.

- **Nothing was profiled.** Every performance claim is derived from reading code and counting call sites. The ranking is by *expected* impact. Phase 2 and 3 should be validated with Dubs Performance Analyzer before and after.
- The draw-call budget in §2.2 is a static count, not a frame capture.
- Whether `RegexOptions.Compiled` is a net loss on this Mono runtime (P10) is *reasoning*, not benchmarking. Do not act on it without a measurement.
- I did not decompile `Dialog_ArchotechInfoView`'s body, so route (a) of A-T1 is unproven.
- I did not verify Dubs Performance Analyzer's public API surface (§3.5) at all.
- I did not enumerate which third-party mods in the 909-mod list call `AddLogWindowWidget` — only that HugsLib itself registers three and the API is public.
- The two-draw card's behaviour at fractional UI scale is a stated risk, not a tested result.